

Multithreading basics

COMPLETE LEARNING GUIDE

What You'll Learn

Multithreading lets a Java program execute multiple sequences of instructions concurrently, sharing the same process memory — essential for responsive applications and for making full use of multi-core hardware, but also the source of some of the hardest bugs in software engineering, because concurrent code can behave differently (or incorrectly) depending on scheduling timing that's invisible in the source code. This guide covers the two ways to create a thread (extending Thread versus implementing Runnable, and why the latter is preferred), the thread lifecycle, the synchronized keyword and Java's intrinsic per-object lock, the wait/notify coordination mechanism, race conditions and why they happen, the volatile keyword's visibility guarantee, and a first look at the Executor framework as the modern alternative to managing raw Thread objects by hand.

1. Creating Threads: extends Thread vs. implements Runnable

A thread of execution can be created by extending Thread and overriding its run() method, or by implementing the Runnable functional interface and passing an instance to a Thread constructor. implements Runnable is preferred almost universally, because Java supports only single inheritance — extending Thread burns your one available superclass slot, whereas implementing Runnable leaves the class free to extend something else, and it also cleanly separates "the task to run" (Runnable) from "the mechanism that runs it" (Thread), which matters once you move to thread pools that don't use raw Thread objects directly at all.

JAVA

```
// Preferred: implements Runnable
Runnable task = () -> System.out.println("Running on: " +
Thread.currentThread().getName());
Thread t = new Thread(task);
t.start(); // NOT t.run() -- calling run() directly executes on the current
thread, no new thread created
```

2. The Thread Lifecycle

A Thread moves through a well-defined set of states, queryable via getState(): NEW (created but start() not yet called), RUNNABLE (executing or eligible to be scheduled by the OS — Java doesn't distinguish "running" from "ready to run" as separate states), BLOCKED (waiting to acquire a lock held by another thread), WAITING / TIMED_WAITING (paused indefinitely or for a bounded time via wait(), join(), or sleep()), and TERMINATED (run() has completed, permanently — a terminated thread can never be restarted; calling start() twice on the same Thread object throws IllegalStateException).

- NEW — created, start() not yet called.
- RUNNABLE — executing or eligible for CPU scheduling.
- BLOCKED — waiting to acquire an object's intrinsic lock.
- WAITING / TIMED_WAITING — paused via wait(), join(), or sleep(), until notified or the timeout elapses.
- TERMINATED — run() has finished; permanent, cannot be restarted.

3. Race Conditions: Why Shared Mutable State Is Dangerous

A race condition occurs when two or more threads access shared mutable state concurrently, and the outcome depends on the unpredictable timing of their execution — the classic example is `count++` on a shared field, which looks atomic but is actually three separate steps (read, increment, write) that can interleave between threads, silently losing updates. With two threads each incrementing a shared counter one million times, the final value is almost never two million without synchronization, because both threads can read the same stale value before either writes back its incremented result.

JAVA

```
class Counter {
    private int count = 0;
    public void increment() { count++; }    // NOT atomic: read, add 1, write --
    public int get() { return count; }
}
// Two threads calling increment() 1,000,000 times each often finish
// with a total LESS than 2,000,000 due to lost updates
```

4. synchronized: Java's Intrinsic Lock

The `synchronized` keyword ensures only one thread at a time can execute a given block of code guarded by the same lock object — every Java object has an intrinsic lock (monitor) built in, and `synchronized` acquires that lock on entry and releases it on exit (even if an exception is thrown), making the guarded section atomic with respect to other threads trying to enter a section synchronized on the same object. A `synchronized` instance method locks on `this`; a `synchronized` static method locks on the class object (`ClassName.class`) — these are different locks, so a `synchronized` static method and a `synchronized` instance method never block each other.

JAVA

```
class Counter {
    private int count = 0;
    public synchronized void increment() {    // acquires 'this' lock before
running    count++;
    }
    public synchronized int get() {
        return count;
    }
}

// Now two threads calling increment() 1,000,000 times each reliably total
2,000,000
```

5. wait(), notify(), and notifyAll()

wait(), notify(), and notifyAll() are methods on Object (not Thread) used for thread coordination and must be called from within a synchronized block holding the same lock the thread will wait/notify on, or an `IllegalMonitorStateException` is thrown. wait() releases the lock and suspends the calling thread until another thread calls notify() (wakes one arbitrary waiting thread) or notifyAll() (wakes all waiting threads) on the same object; the classic producer-consumer pattern uses this to make a consumer thread wait when a shared queue is empty and a producer thread notify it once an item is added. Because a woken thread doesn't necessarily get to run immediately, wait() should always be called inside a while loop re-checking the condition, not a single if — a pattern known as guarding against "spurious wakeups."

6. volatile: A Visibility, Not Atomicity, Guarantee

The volatile keyword guarantees that reads and writes to a field go directly to main memory rather than being cached in a CPU register or a per-thread cache, so every thread always sees the most recently written value — this solves visibility problems (a flag set by one thread not being seen by another) but does NOT make compound operations like count++ atomic, since volatile provides no mutual exclusion at all. volatile is the right tool for a simple flag one thread sets and others poll (e.g. a shutdown signal), but the wrong tool for anything involving a read-modify-write sequence, which still needs synchronized or an Atomic class.

7. A First Look at the Executor Framework

Manually creating and managing raw Thread objects doesn't scale — threads are expensive to create, and an application that spins up a new Thread per task can exhaust system resources under load. The Executor framework (`java.util.concurrent`) decouples task submission from thread management: `ExecutorService` manages a reusable pool of worker threads, and `submit(Runnable)` or `submit(Callable)` hands it a task to run on whichever pooled thread becomes available, amortizing thread-creation cost across many tasks. This is covered in full depth in the Advanced Concurrency submodule, but it's worth knowing from the start that production Java code almost never creates raw Thread objects directly — it goes through an `ExecutorService`.

JAVA

```
ExecutorService pool = Executors.newFixedThreadPool(4);  
pool.submit(() -> System.out.println("Task running on a pooled thread"));  
pool.shutdown(); // stop accepting new tasks, let submitted ones finish
```

Common Pitfalls

- Calling `thread.run()` directly instead of `thread.start()` — this executes the code synchronously on the current thread, never actually starting a new thread.
- Treating `count++` as atomic and skipping synchronization, silently losing updates under concurrent access (a classic race condition).
- Calling `wait()/notify()` outside a synchronized block on the same lock object, throwing `IllegalMonitorStateException`.
- Using `volatile` to try to make a read-modify-write operation (like an increment) thread-safe — it only guarantees visibility, not atomicity.
- Checking a `wait()` condition with `if` instead of a `while` loop, missing the possibility of a spurious wakeup where the condition isn't actually true yet.
- Synchronizing on different lock objects (e.g. one method locks on this, another on a private field) when both are meant to protect the same shared state, providing no real mutual exclusion.
- Creating a new raw `Thread` per task in a hot path instead of using a pooled `ExecutorService`, exhausting system resources under load.

Key Takeaways

- Prefer implements `Runnable` over extends `Thread` — it avoids burning Java's single-inheritance slot and separates the task from the execution mechanism.
- A race condition arises when unsynchronized threads read/modify/write shared state — even a simple `count++` is not atomic.
- `synchronized` provides both mutual exclusion (only one thread in the block at a time) and visibility (changes are flushed to main memory on unlock).
- `volatile` guarantees visibility only, never atomicity — it cannot replace `synchronized` for compound operations.
- `wait()/notify()/notifyAll()` require holding the object's lock and should always re-check their condition in a `while` loop, not an `if`.
- Production code manages threads through an `ExecutorService` thread pool rather than creating raw `Thread` objects directly.

Learn More

Oracle Java Tutorials — Concurrency

<https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html>

Oracle Java Tutorials — Guarded Blocks (wait/notify)

<https://docs.oracle.com/javase/tutorial/essential/concurrency/guardmeth.html>

Baeldung — Guide to the volatile Keyword

<https://www.baeldung.com/java-volatile>